

CameraIa: An Edge-Side, Browser-Native Framework for Remote Psychophysiological Experiments

Kotaro Furukawa^{*1}

Abstract – CameraIa is an edge-side, browser-native framework for remote psychophysiological experiments. It synchronizes behavioral task events with face-image-derived records and exports local CSV/ZIP files without transmitting or storing raw facial video. We examined an exploratory N=1 author-participant deployment comprising 10 sessions and 600 trials in laboratory and home settings. Under the tested configuration, all planned sessions and trials were completed, and timestamped task-event and face-image-derived records were exported using the same browser clock domain. The exported records showed descriptive laboratory/home differences in facial ROI green-channel values, head-motion estimates, and a frame-to-frame green-channel change indicator; these were interpreted only as deployment-context differences. The findings are limited to the tested participant and configuration and do not establish validated physiological measurement or general performance across users, devices, and environments.

Keywords : Browser-Native Sensing, Remote Photoplethysmography, FaceMesh, Edge-Side Processing, Task-Aligned Measurement, Laboratory and Home Settings

1. Introduction

Remote psychological and psychophysiological task experiments require behavioral task events and webcam-derived records to be related in time. When such experiments are conducted in a participant’s home or another private space, retaining or transmitting identifiable raw facial video also creates a data-handling burden. Browser-side processing can address this practical concern by deriving task-relevant records on the participant’s device and persisting only the resulting fields.

Browser-based task platforms already support remote behavioral experiments^{[10], [11]}, and browser-, web-, and edge-based rPPG systems have also been reported^{[4]~[7]}. The contribution sought here is therefore not a new rPPG estimator. The implementation challenge addressed by CameraIa is to combine task execution, client-side face-derived recording, shared-clock timestamps, and local persistence within one browser runtime for a remote task experiment.

To address this challenge, we present **CameraIa**, an edge-side, browser-native framework for remote psychophysiological experiments. Here, edge-side means that webcam frames are processed within the participant’s browser on the client device. Dur-

ing task execution, CameraIa records task events and face-image-derived fields on the same browser clock and exports them locally as CSV files in a ZIP archive, without transmitting or retaining raw facial video. Processing during task execution supports feature-only local logging and avoids a separate server-side timestamp-reconciliation stage.

The scope of this study is a seated browser task conducted in laboratory and home settings and evaluated as a system-integration implementation. The participant views stimuli on a laptop and responds by key press; the study does not address continuous sensing during ordinary daily activities. The recorded facial ROI color traces are intended for possible subsequent rPPG analysis, but the present evaluation does not validate a heart-rate waveform or estimate heart-rate accuracy.

1.1 Research Questions

The exploratory Research Questions (RQs) focus on procedure completion, exported records, and descriptive between-environment observations under the tested configuration:

- **RQ1:** Under the tested configuration, did CameraIa complete all planned sessions and trials and export timestamped task-event and face-image-derived records using the same browser clock domain?
- **RQ2:** What between-environment differ-

^{*1}: College of Transdisciplinary Sciences for Innovation, Kanazawa University, Japan

ences are observable in the task-synchronized face-image-derived and sensor-derived exported records?

For RQ1, the planned procedure comprised 10 sessions of six 10-trial blocks, yielding 600 trials in total. Task, block, and exported frame records were timestamped with `performance.now()` in the same client-side browser runtime. Reaction times of 100–1500 ms were used separately as a study-specific preprocessing range for descriptive trial-level summaries, not as a criterion for system completion or timestamp accuracy. For RQ2, the exported records included facial ROI green-channel values, motion estimates, and a frame-to-frame green-channel change indicator. Laboratory/home differences in these records were interpreted as mixed deployment-context differences because lighting, screen light, posture, camera geometry, task timing, the measurement pipeline, and participant state were not independently separated.

1.2 Contributions

In this paper, we focus on system integration and conservative deployment observations. The contributions are:

1. **Edge-side browser-native system integration:** An implementation for remote psychological and psychophysiological task experiments that combines task execution, face-derived feature extraction, timestamp alignment, and local persistence within one browser runtime.
2. **Task-synchronized feature logging pipeline:** A logging pipeline that records task events, facial ROI green-channel values, FaceMesh-derived motion estimates, and frame-derived green-channel change indicators without raw facial video persistence.
3. **Exploratory deployment under limited conditions:** An N=1 author-participant deployment documenting planned-procedure completion and descriptive exported-record differences under laboratory/home conditions.

The remainder of this paper is organized as follows. Section II reviews related work in rPPG, web/edge physiological sensing, and task-measurement platforms. Section III describes the Camerala system architecture. Section IV describes the browser-based task deployment and reporting details. Section V presents descriptive observations. Section VI dis-

cusses design requirements and safeguards for future deployments, and Section VII concludes.

2. Related Work

2.1 Remote Physiological Measurement and rPPG

Remote photoplethysmography (rPPG) estimates pulse-related color changes from facial video. Classic approaches include ambient-light remote plethysmographic imaging and blind-source video methods^{[1],[2]}, while more recent work includes deep-learning-based estimators^[3]. These studies show that camera-based and webcam-based rPPG functionality already exists. They also show that uncontrolled capture conditions, implementation choices, and evaluation pipelines are important factors in remote heart-rate imaging. Accordingly, this work uses facial ROI color traces as task-aligned exported records rather than proposing a new rPPG estimator.

2.2 Browser-Based, Web-Deployable, Edge-Based, and Privacy-Oriented rPPG Systems

Several closely related prior studies and systems are directly relevant to Camerala's positioning. Ayesha *et al.* proposed a web application for experimenting with and validating remote measurement of vital signs^[4]. LabVanced provides remote heart-rate detection/rPPG functionality within an online experiment platform^[5]. FacePhys-Demo demonstrates browser-based rPPG monitoring with local inference in a public web demo/repository^[6]. RhythmEdge addresses contactless heart-rate estimation on edge devices^[7]. Gupta and Etemad investigated privacy-preserving remote heart-rate estimation from facial videos^[8]. Di Lernia *et al.* evaluated rPPG via online webcams under uncontrolled conditions^[9]. Together, these related studies and public systems indicate that browser-based, web-deployable, edge-based, and privacy-oriented rPPG have already been explored. Camerala is positioned as a system-integration implementation for remote psychological and psychophysiological task experiments: it combines browser-based task execution, face-derived feature extraction, shared-clock task/frame timestamps, and local CSV/ZIP persistence without raw facial video retention within one browser runtime.

2.3 Task Measurement and Physiological Data Collection Platforms

Well-established web experiment libraries and experiment runtimes, such as jsPsych^[10] and PsychoPy/PsychoJS/Pavlovia^[11], support browser-based task execution and online experiments. Although such integrations can be constructed, these libraries focus primarily on task execution, leaving webcam processing and client-side feature persistence deployment-specific. Conversely, cloud analytics APIs can provide rich video-based analysis but often require transmitting video streams away from the participant’s device. Table 1 maps this design trade-off space.

We structure the initial evaluation as a single-case exploratory deployment that documents whether the integrated browser runtime can complete the planned browser-based task logging procedure and what differences appear in the exported records from the tested laboratory and home contexts.

3. System Architecture: Camerale

Camerale (derived from "Camera" + "Laboratory") is an edge-side, browser-native research implementation for remote psychological and psychophysiological task experiments. It integrates task execution, face-derived feature extraction, shared-clock task/frame timestamps, and local CSV/ZIP persistence without raw facial video retention within one browser runtime. Although the deployment in this paper used one Gabor-orientation task, the runtime separates task logic from the sensing pipeline so that other task procedures can be implemented without changing the feature-extraction loop.

3.1 High-Level Architecture

The system enables a browser-based task experimental loop inside the browser sandbox (Fig. 1). The architecture consists of three coupled modules:

1. **Task Engine:** Stimulus rendering, trial logic, key responses, feedback, staircase updates, and reaction-time logging.
2. **Vision Pipeline:** Webcam ingestion, MediaPipe Face Mesh inference, facial ROI green-channel extraction, head-motion estimates, blink/EAR features, and frame-derived fields.
3. **Data Manager:** Shared-clock timestamping, trial/frame aggregation, CSV serialization, ZIP packaging, and local persistence without raw-

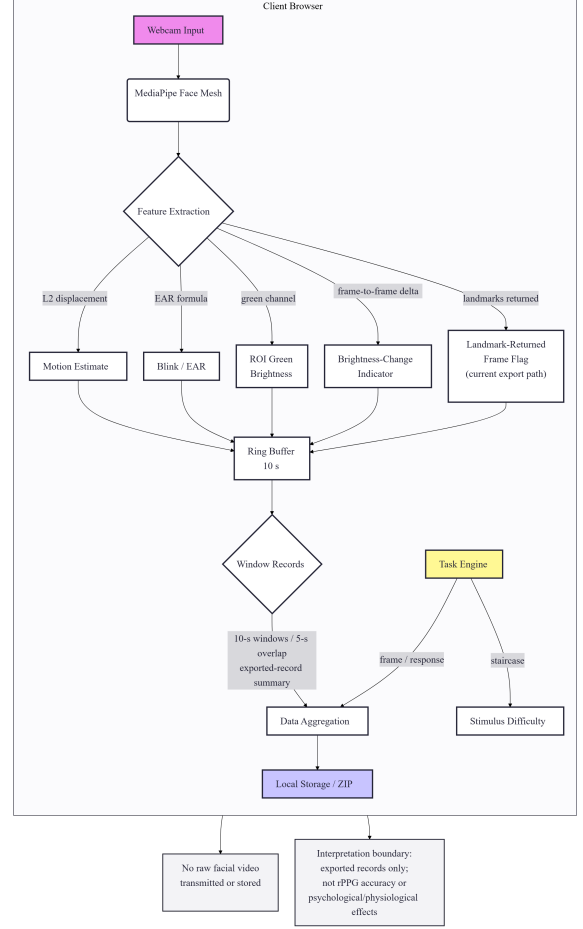


Fig. 1 Overview of the Camerale system architecture. The browser obtains webcam frames, extracts face-derived exported fields, aggregates 10-s window records, synchronizes them with task events, and exports local CSV/ZIP logs. Under the tested implementation, only extracted fields and task events are logged; raw facial video is neither transmitted nor stored by Camerale.

video storage.

3.2 Technical Implementation: The Vision Pipeline

The current implementation uses MediaPipe Face Mesh as a software framework for browser-based facial landmark extraction^[12]. The camera stream is requested through `getUserMedia` with ideal constraints of 640×480 pixels and 30 fps. Because actual camera settings can depend on the browser and device, these constraints should be interpreted as requested settings rather than verified hardware settings.

Table 1 Design Trade-off Mapping of Data Collection Platforms

System / Platform	On-device processing	video	Task-sensing synchronization	syn-	Client-side persistence	per-	Raw export	video	Typical use
Web experiment libraries	N/A (task only)		Partial		Depends on setup		N/A		Behavioral tasks
Experiment runtimes	Depends on deployment		Yes for task events		Depends on setup		Depends		Lab/online experiments
Browser or web rPPG systems	Often possible		Varies by platform		Varies		Varies		Vital-sign estimation
Cloud sensing APIs	No		Low or asynchronous		No		Often required		Video analytics
Camera	Client-side feature extraction	feature	Task events and frame features one runtime	and	Local ZIP export of logs		Not used by Camera		Remote task studies

Note: The table summarizes design emphasis rather than categorical superiority. Capabilities depend on implementation and deployment configuration.

3.2.1 Feature Extraction Logic

The present implementation records face-image-derived and sensor-derived fields for descriptive analysis. Two categories are central to this paper:

1. Head Motion Index (M_t) Head motion is approximated from the frame-to-frame displacement of a facial landmark. Let $L^t = (x^t, y^t)$ be the normalized landmark coordinate at frame t . The motion scalar is computed as:

$$M_t = \|L^t - L^{t-1}\|_2. \quad (1)$$

This feature is retained as a motion-related exported field for descriptive inspection of task-synchronized records. It is not an algorithmic correction for motion artifacts in rPPG and is not used to compensate for motion-induced rPPG degradation.

2. ROI Green-Channel Value and Brightness-Change Indicator The implementation samples a small forehead region around a FaceMesh landmark and records its green-channel intensity as a facial ROI value intended for possible subsequent rPPG analysis. It also computes a frame-to-frame green-channel brightness-change indicator:

$$B_\Delta(t) = |G_t - G_{t-1}|, \quad (2)$$

where G_t is the downsampled frame-level green-channel average at frame t . Camera records this field together with the motion estimate for descriptive inspection of exported records. The brightness-change indicator does not measure camera exposure settings, external illuminance, specific environmental events, psychological or physiological state, or rPPG accuracy.

3.3 Technical Implementation: The Task Engine

The experimental psychophysical probe is a two-alternative forced-choice orientation discrimination

task. Using the HTML5 Canvas API, the task engine procedurally renders oriented grating stimuli, records key responses, calculates reaction time with `performance.now()`, and updates task difficulty. Procedural rendering avoids network-dependent stimulus loading during a trial and keeps task timing in the same browser runtime as feature logging.

3.3.1 Timing and Synchronization

The implementation uses the following timing strategy:

- 1. High-resolution timestamps:** Trial, block, and frame timestamps use `performance.now()`; the RT reference is captured immediately after the synchronous Canvas draw call returns.
- 2. Frame-loop alignment:** Feature extraction and task rendering are coordinated with the browser frame loop.
- 3. Local runtime alignment:** Task events and sensor-derived features are logged in the same browser runtime, avoiding server-side reconciliation.

3.4 Data Management and Client-Side Persistence

The current implementation uses JSZip to export local log files as a ZIP archive. The archive contains `metadata.json`, `trials.csv`, `features_raw.csv`, `windows.csv`, `blocks.csv`, and `subjective.csv`. The metadata file stores the session ID, start time, user agent, and block plan. The user initiates the download at the end of the session. This design avoids Camera-side raw-video storage and avoids raw-video transmission, but it also reduces the ability to inspect image artifacts retrospectively. For this reason, Camera logs reproducible exported fields such as facial ROI green-channel values, head-motion estimates, and a frame-to-frame green-

channel change indicator.

4. Methodology

4.1 Target Usage Scenario and Exploratory Deployment

The deployment was a browser-based task experiment conducted in one laboratory setting and one home setting. The participant sat in front of a laptop, viewed Gabor stimuli, and responded using the keyboard. This setting should be distinguished from unconstrained daily-life physiological sensing during walking, conversation, cooking, sleep, or other ordinary activities.

To evaluate operational completion under this scenario, we conducted an exploratory longitudinal deployment with one author-participant. The deployment consisted of 10 sessions and 600 total trials. Each session consisted of six 10-trial blocks, with two blocks for each instructional task context (neutral instruction, negative instruction, and positive instruction), as reflected in the exported block plans. The order of blocks was randomized by the web application.

4.2 Participant (Autoethnography)

The participant was the primary author. This was a self-experimentation evaluation: no other participants were involved, no identifiable raw video was transmitted by CameraLa, and the author provided informed consent for self-participation. Because the participant was also the author and knew that the task instructions were experimentally constructed, the instruction blocks are used only as exploratory task-interface contexts for demonstrating synchronized logging.

4.3 Hardware and Software Configuration

Table 2 summarizes the hardware and software information available for the deployment. Values unavailable from the deployment records are explicitly marked as not logged.

4.4 Deployment Environment and Lighting Conditions

The deployment was conducted during daytime in an indoor university laboratory/research room and an indoor private home room under general artificial room lighting. Both rooms had windows, but curtains were closed during the sessions. Direct natural light was not used as the primary illumination source in either context. The built-in laptop dis-

Algorithm 1 CameraLa Task Loop (Simplified)

```

Initialize Camera, FaceMesh, Canvas
StartContinuousVisionFrameLoop()
while Session Active do
  Block  $\leftarrow$  GetCurrentBlock()
  Context  $\leftarrow$  Block.instruction
  ShowInstruction(Context.text)
  for trial  $i = 1$  to  $n$  do
     $t_{trial} \leftarrow$  performance.now()
     $\theta \leftarrow$  SampleOrientation( $\{-45^\circ, +45^\circ\}$ )
    ShowFixation(400–600 ms)
    IssueCanvasDrawCommands( $\theta$ )
     $t_{ref} \leftarrow$  performance.now()
    while No KeyPress do
      Poll F/J KeyPress
    end while
     $t_{key} \leftarrow$  performance.now()
     $RT \leftarrow t_{key} - t_{ref}$ 
    LogTrialRecord( $t_{trial}$ , Key,  $RT$ , Accuracy, Context)
    UpdateStaircase(Accuracy, relative  $\pm 5\%$  contrast step)
  end for
end while

```

Table 2 Hardware and software configuration

Item	Reported value
Laptop/PC model	mouse Computer G-Tune P517G60ZO21CNHWT
CPU	Intel Core i7-13620H, 10 cores (P6+E4), 16 threads
GPU	NVIDIA GeForce RTX 5060 Laptop GPU
RAM	32 GB
Storage	1 TB SSD
Operating system	Windows 11 Home 64-bit; exact deployment-time build not logged
Display	15.6-inch display, 2560×1440 pixels, 165 Hz
Browsers used	Mozilla Firefox and Google Chrome
Browser versions	Firefox user-agent was available when recorded in exported metadata; exact Chrome deployment-time version was not systematically logged
Webcam type	Built-in laptop webcam
Webcam model	Exact internal webcam model not logged
Camera capture API	Browser <code>getUserMedia</code> API
Requested camera constraints	Ideal 640×480 pixels, ideal 30 fps
Actual camera resolution	Browser/device-negotiated; not programmatically logged as a verified deployment-time value
Actual camera frame rate	Browser/device-negotiated; not programmatically logged as a verified camera frame rate
Screen brightness	Fixed during the deployment; exact percentage not logged
Auto-exposure	Device/browser default behavior; not manually constrained and exact state not programmatically logged
Auto-white-balance	Device/browser default behavior; not manually constrained and exact state not programmatically logged

play brightness was fixed across sessions. No external illuminance measurement was performed. Table 3 summarizes the deployment environment and lighting conditions.

4.5 Experimental Procedure and Instructional Task Contexts

Each session began with a pre-session capture check for basic camera framing and pose before the camera feed was hidden to reduce distraction. The task included negative, neutral, and positive instruction contexts as exploratory task-interface conditions. These contexts were used to demonstrate that Cameralea can present different instruction frames while recording synchronized behavioral and face-derived records. The labels “negative” and “positive” refer only to instruction labels used in the task interface.

4.6 Psychophysical Task

The task was a two-alternative forced-choice orientation discrimination task using procedurally rendered Gabor stimuli. Each trial began with a randomized fixation period of 400–600 ms, followed by a Gabor stimulus oriented at either $+45^\circ$ or -45° for 200 ms. The participant responded using the F/J keys: the F key corresponded to -45° , and the J key corresponded to $+45^\circ$. Exported trials stored browser-relative `startTime`, response, RT,

correctness, and task fields but not separate onset/response timestamps; face-derived records used the same browser clock. To maintain adaptive task difficulty, the implementation used a standard 1-up 1-down staircase procedure on stimulus contrast. Stimulus contrast was adjusted by relative $\pm 5\%$ steps according to the previous response. These task parameters are reported for reproducibility.

4.7 Dependent Measures and Pre-processing

All exported logs were processed sequentially. The primary behavioral measure was reaction time (RT), with an operational analysis range of 100–1500 ms. The primary sensor-derived measures were facial ROI green-channel values, head-motion estimates, and the frame-to-frame green-channel change indicator. Behavioral trials were included in descriptive task analyses when reaction times satisfied $100 \leq RT \leq 1500$ ms. Cameralea used MediaPipe FaceMesh to obtain facial landmarks in the browser. Task records and frame records were timestamped with `performance.now()` within the same browser runtime during active sessions.

5. Results

The results are reported descriptively under the tested configuration. For RQ1, all 10 planned sessions and 600 trials were completed,

Table 3 Deployment environment and lighting conditions

Item	Laboratory	Home
Room type	Indoor university laboratory/research room	Indoor private home room
Time of day	Daytime	Daytime
Primary lighting	General indoor artificial lighting	General indoor artificial lighting
Window presence	Windows present	Windows present
Curtain/blind condition	Curtains present and closed	Curtains present and closed
Natural light contribution	Direct natural light was not used as the primary illumination source	Direct natural light was not used as the primary illumination source
External illuminance measurement	No external illuminance measurement was performed.	
Screen brightness	Fixed across sessions using the built-in laptop display setting.	
Task display	Stimuli were presented on the built-in laptop display. Display output was part of the tested configuration rather than an independently manipulated factor.	
Window aggregation	Exported window-level features used 10-s windows with 5-s overlap.	
Brightness-change indicator aggregation	The frame-to-frame green-channel brightness-change indicator was summarized in 10-s windows with 5-s overlap.	
Clouds/screen flicker/power cycles	Not treated as observed experimental events; possible brightness variation was handled only as camera-image-derived record variation.	

and timestamped task-event and face-image-derived records were exported for every session. Trial, block, and frame timestamps were generated with `performance.now()` in the same client-side browser runtime and therefore used the same browser clock domain.

For descriptive RT summaries, 541/600 trials (90.2%) were included after applying the study-specific 100–1500 ms preprocessing range; the counts were 256/300 in the laboratory setting and 285/300 at home. This preprocessing range was not used as a criterion for procedure completion or timestamp accuracy.

5.1 Task and Exported Feature Summaries

Table 4 summarizes task-level and exported-record values in the laboratory and home contexts. After RT exclusion, mean RT was 721.1 ± 257.7 ms in the laboratory setting and 676.5 ± 249.5 ms in the home setting. Accuracy was 85.5% and 82.1%, respectively. Window-level exported records showed descriptive differences in facial ROI green-channel values, motion estimates, and the green-channel change indicator between settings. Because lighting, screen light, posture, camera geometry, task timing, and participant state were not independently controlled, these differences are reported only as deployment-context differences in exported records.

5.2 Between-Environment Exported-Record Differences

To address RQ2, we compared descriptive task-synchronized records between the laboratory and home contexts. The exported records showed differences in facial ROI green-channel values, motion estimates, and the frame-to-frame green-channel change indicator. Because lighting, screen light, posture, camera geometry, task timing, and participant state were not independently controlled, these differences are reported only as deployment-context differences in exported records. Motion estimates are reported as exported face-derived records for descriptive comparison between the laboratory and home contexts.

The instruction contexts are reported only to document operation of the logging pipeline across different task-interface contexts. In the laboratory context, the negative instruction block showed a lower mean RT than the neutral block in this dataset. In the home context, the mean RT values across instruction blocks were closer together. Because the deployment used one author-participant and did not isolate instruction effects from deployment factors, these RT patterns are descriptive task-event observations rather than evidence of an instruction-induced effect.

5.3 Facial ROI Green-Channel Records

The exported data include facial ROI green-channel values intended for possible subsequent rPPG analysis, but no ECG or contact PPG ref-

Table 4 Descriptive task and exported-record summaries; RT summaries use the study-specific 100–1500 ms preprocessing range.

Metric	Laboratory	Home	Notes
Trials included in descriptive RT summaries after applying the study-specific range	256/300 (85.3%)	285/300 (95.0%)	Trial-level; RT 100–1500 ms
Mean RT within RT range	721.1 $\pm$ 257.7 ms	676.5 $\pm$ 249.5 ms	Trial-level included trials
Accuracy within RT range	85.5%	82.1%	Trial-level included trials
Facial ROI green-channel value	46.69 $\pm$ 6.80	34.32 $\pm$ 2.72	Window-level; windows.csv
Motion estimate	0.0011 $\pm$ 0.0005	0.0010 $\pm$ 0.0004	Window-level; windows.csv
Frame-to-frame green-channel change indicator	0.0504 $\pm$ 0.0509	0.0707 $\pm$ 0.0337	Window-level; windows.csv

erence was collected. These values are therefore reported as face-image-derived records rather than validated heart-rate changes.

6. Discussion

This limited deployment identifies practical requirements for future edge-side browser-native logging systems: shared-clock task/frame timestamps, feature-only local persistence, hardware/environment reporting, and cautious interpretation when raw facial video and reference physiological sensors are not retained. Under the tested configuration, Cameralea completed the planned sessions and trials and exported task-event and face-image-derived records from the same browser clock domain. The laboratory/home values answer RQ2 only as descriptive differences in exported records; they combine environmental, camera, task, and participant factors that were not independently separated.

Cameralea’s useful system property is integration: task timing, frame-level feature extraction, and local export are handled in one client-side workflow. This can reduce raw-video handling and server-side timestamp reconciliation, but it also removes the possibility of visually inspecting many artifacts after data collection. The retained facial ROI, head-motion, and green-channel change fields support descriptive inspection, while physiological validation requires a reference sensor and a larger evaluation.

The laboratory/home differences observed in this deployment should be interpreted as mixed deployment-context differences that may reflect environmental conditions, screen light, posture, camera geometry, the measurement pipeline, task timing, participant state, or their mixture. Future deployments should therefore incorporate shared-clock task/frame timestamps, documented hardware/browser/camera settings, fixed screen-

brightness settings, pre-session camera/lighting checks, motion and camera-derived green-channel change records, environment reporting, and predefined artifact-handling rules. Future evaluations should log the actual negotiated camera resolution and effective frame rate, together with relevant camera-control settings such as auto-exposure and auto-white-balance, to improve reproducibility and interpretation. If privacy-sensitive raw video is not stored, these safeguards become more important because many artifacts cannot be retrospectively inspected from images; for studies that aim to make claims about physiological responses, a reference physiological sensor and a multi-participant design are necessary.

6.1 Limitations

This exploratory N=1 author-participant deployment used one primary laptop/camera configuration, no ECG or contact PPG reference, and no comparison with existing experiment runtimes, browser rPPG tools, or cloud APIs. It did not test multiple participants, devices, or home environments, and it did not experimentally separate lighting, screen light, posture, camera placement, task timing, participant state, and task context. The normal export path stored frame records only when FaceMesh landmarks were returned; therefore, the saved records cannot be used to estimate an overall landmark-detection success rate across all camera frames. RT used browser-side `performance.now()`; physical onset was not externally verified and separate onset/response timestamps were not exported, precluding exact frame-interval matching. Future work should use reference physiological sensors, controlled posture/camera conditions, predefined artifact-handling rules, and logging paths that retain detection outcomes for all processed frames. The study is limited to a system-integration example of browser-native, task-aligned, client-side fea-

ture logging under the tested laboratory and home conditions.

7. Conclusion

This paper presented CameraLa as an edge-side, browser-native framework for remote psychophysiological experiments. In the tested N=1 deployment, CameraLa completed all 10 sessions and 600 trials and exported timestamped task-event and face-image-derived records using the same browser clock domain. The between-environment observations are limited to descriptive differences in exported records, and the implementation evidence does not establish validated physiological measurement or general performance across other users, devices, and environments.

Declarations

Data availability. The dataset cannot be made publicly available because it contains privacy-sensitive face-derived records from a domestic deployment context.

CRedit authorship contribution statement.

Kotaro Furukawa: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing - original draft.

Declaration of competing interest. The author declares no competing interests.

Funding. This work was supported by the College of Transdisciplinary Sciences for Innovation, Kanazawa University.

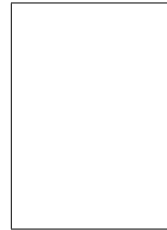
Reference

- [1] Wim Verkruysse, Lars O. Svaasand, and J. Stuart Nelson. Remote plethysmographic imaging using ambient light. *Optics Express*, 16(26):21434–21445, 2008.
- [2] Ming-Zher Poh, Daniel J. McDuff, and Rosalind W. Picard. Non-contact, automated cardiac pulse measurements using video imaging and blind source separation. *Optics Express*, 18(10):10762–10774, 2010.
- [3] Weixuan Chen and Daniel McDuff. Deepphys: Video-based physiological measurement using convolutional attention networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 349–365, 2018.
- [4] A. H. Ayesha, D. Qiao, and F. Zulkernine. A web application for experimenting and validating remote measurement of vital signs. In *Information Integration and Web Intelligence (IIWAS 2022)*, Lecture Notes in Computer Science, vol. 13635, pages 237–251, 2022. doi:10.1007/978-3-031-21047-1_21.
- [5] LabVanced. Remote heart rate detection / rPPG. <https://www.labvanced.com/content/technology/en/remote-heart-rate-detection-rPPG/>, accessed July 5, 2026.
- [6] K. Wang. FacePhys-Demo: Browser-based rPPG monitor with local inference. <https://github.com/KegangWangCCNU/FacePhys-Demo>, accessed July 5, 2026.
- [7] Z. Hasan, E. Dey, S. R. Ramamurthy, N. Roy, and A. Misra. Demo: RhythmEdge: Enabling contactless heart rate estimation on the edge. In *2022 IEEE International Conference on Smart Computing (SMART-COMP)*, pages 177–179, 2022.
- [8] D. Gupta and A. Etemad. Privacy-preserving remote heart rate estimation from facial videos. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 706–712, 2023. doi:10.1109/SMC53992.2023.10394350.
- [9] D. Di Lernia, G. Finotti, M. Tsakiris, G. Riva, and M. Naber. Remote photoplethysmography (rPPG) in the wild: Remote heart rate imaging via online webcams. *Behavior Research Methods*, 56:6904–6914, 2024. doi:10.3758/s13428-024-02398-0.
- [10] J. R. de Leeuw, R. A. Gilbert, and B. Luchterhandt. jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, 8(85):5351, 2023. doi:10.21105/joss.05351.
- [11] J. W. Peirce, J. R. Gray, S. Simpson, M. MacAskill, R. Hoechenberger, H. Sogo, E. Kastman, and J. K. Lindelov. PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51:195–203, 2019. doi:10.3758/s13428-018-01193-y.
- [12] Google AI Edge. MediaPipe Face Landmarker. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker, accessed July 5, 2026.

(received April 26, 2026, revised July 17)

Biography

Kotaro Furukawa



Kotaro Furukawa is with the College of Transdisciplinary Sciences for Innovation, Kanazawa University, Japan.